Converge

THE CANONICAL BLUEPRINT · V4 — FORK COLLAPSED · THE CVG CLI · SIX-TIER TASKING

The Eight Passes — Compile Intent, *Close the Loop.*

The canonical blueprint for compiling a fuzzy brief — or a raw idea — into an autonomous, eval-gated system — **eight passes, one on-ramp, and one path** to a closed loop that drives a fleet to green. The fork is **collapsed**: consensus always hands off to tasking, and one six-tier engine sizes every unit from a one-liner to a whole backbone. A referee CLI — **cvg** — gates every pass. Build the factory from day one; walk out one gate at a time; teach every pass as you go.

8	13	6	cvg
PASSES · ONE PATH	CLAUDE CODE SKILLS	TASK SIZES XS → XXL	REFEREE CLI V0.10.0

DOCUMENT
Canonical process blueprint · v4 (fork-collapsed)

CONVERGED MEANS
an eval passed — not that you feel done

The invariant — every pass lowers altitude, binds an engine, ends at a gate. Pass 8 is the exception: it does not lower, it closes. v4 records the collapse of the plan-vs-task fork into a single tasking path, the six-tier task-spec engine (XS/S/M/L leaves + XL/XXL decomposition nodes), and the `cvg` CLI — a referee that dispatches engines and gates their output, now at 2026 agent-native SOTA (`--json` envelope, `agent-context` manifest, `tasks dod`). No pass is closed until its lesson exists.

Introduction

Why Converge exists

The old loop ran in your head. A requirement arrived fuzzy, and you held the definition of "done" as private judgment — refined by taste, defended in review, never written down. That loop does not scale to AI engines, because an engine cannot execute a standard it cannot read. The bottleneck was never typing speed; it was that intent lived in a human and had to be re-supervised at every step. The AI-native engineer inverts this: take a fuzzy requirement and lower it through a repeatable descent — each step more precise than the last — until "done" is a machine-checkable condition an engine can satisfy unsupervised.

The main idea — compile intent, don't write the system

Stop writing the system. Compile it. Converge treats building as a **compilation pipeline** that lowers altitude in fixed stages: intent → structure → plan → tasks → code → loop. Each stage is a transformation with a typed output and a gate that must pass before the next stage runs — exactly how a compiler lowers source through intermediate representations down to machine code. You author the high-level intent and the checks; the pipeline emits the system. Your role shifts from author to compiler — you specify the descent and verify each lowering, rather than hand-writing the final artifact. When no client brief exists, an optional on-ramp — **Pass 0 · Capture** — compiles the raw idea into the BRD that starts the descent.

The thesis — build the factory from day one

You do not transform a workflow into a factory later. That retrofit — bolting evals, harnesses, and loops onto human-centric code never built to be gated — is the slow, expensive path, and it is the path almost everyone takes. Converge refuses it. The framework is factory-shaped from Pass 1: every pass already ends in a gate, every task is born with a runnable eval, and the harness is a control plane from the first file. So when you decide to step out, there is nothing to retrofit and nothing to rewrite; the gates already hold without you, and you walk out one gate at a time. And the factory teaches as it builds: understanding is a deliverable equal to working code, so every canonized pass closes with a lesson.

EVERY PASS

Lowers altitude

intent → structure → plan → tasks → code → loop. Each step closer to something a machine runs.

BINDS

An engine

the right tool for the altitude — Cowork to think, Claude Code to build, an adversary to attack, a fleet to crank.

ENDS AT

A gate

converged means an eval passed — not that you feel done. The gate is the unit of trust.

THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate. Pass 8 is the exception — it does not lower, it **closes**. You are converged when the eval passes — not when you feel done. And in the operating model, no pass is closed until its lesson exists.

Contents

The Whole Method, On One Line

`Capture⁰ → Intent → Structure → Decompose → Consensus → Tasking → Register → Harness → Loop`

A raw idea becomes a signed BRD, the BRD becomes a tech-spec, the tech-spec becomes ADRs, the ADRs become swimlane plans, the plans survive an adversarial review — and then, always, decompose into task-specs. The old plan-vs-task fork is **collapsed**: a six-tier engine sizes every unit from a one-liner to a whole backbone, so there is no separate spec-driven paradigm to escape to. Each output is the next input; the chain descends from a fuzzy brief to a fleet running green — and every canonized pass is taught back to the owner before the descent continues.

#	PASS	SKILL	OUT · GATE
0	**Capture** (optional)	`idea-to-brd`	signed BRD — or a no-go record
1	**Intent**	`brd-docs-to-tech-req`	tech-spec · answers the brief
2	**Structure**	`tech-req-to-adrs`	`docs/adrs/*` + `CONTEXT.md` · system understood
3	**Decompose**	`reqs-to-swimlane-plans`	swimlane plans · plan-altitude only
4	**Consensus**	`sketch-plans-adversarial-review`	plans sharpened · `FORK: B` (task-driven)
5B	**Tasking**	`task-spec` (six-tier)	XS/S/M/L leaves · XL/XXL nodes · each has an eval
①	**Register**	`task-specs-to-issues`	1 spec = 1 issue · the board is state
6	**Harness**	`stack-to-harness`	`.claude/` + `AGENTS.md` · control plane
8	**The Loop**	`task-loop --issue N`	each task → green eval → PR

GUARDRAILS MAKE THE SEQUENCE CANONICAL

Pass 0 never touches the repo — idea in, brief out, and Pass 1 must not consume an unsigned brief. Pass 2 writes ADRs only (no code). Pass 3 is plan-altitude only (no tasks, no SQL). Pass 4 always hands off to tasking — the fork is a constant now (`FORK: B`), enforced by the consensus gate. And Pass 8 does not lower — it closes. No pass is closed until its lesson exists.

The spine, drawn — eight passes, one on-ramp, one path

THE FULL DESCENT, NOW A STRAIGHT LINE

The whole descent is now linear. An optional Capture on-ramp feeds it when no brief exists; four passes lower altitude; at Consensus the plans are hardened and the path **always** hands off to Tasking — the plan-vs-task fork is collapsed, because the six-tier task-spec engine absorbs the whole range a separate spec-driven paradigm used to cover. From Tasking the line runs on through Register, Harness, Execute, and the Loop. No branch, no rejoin — one spine, top to bottom.

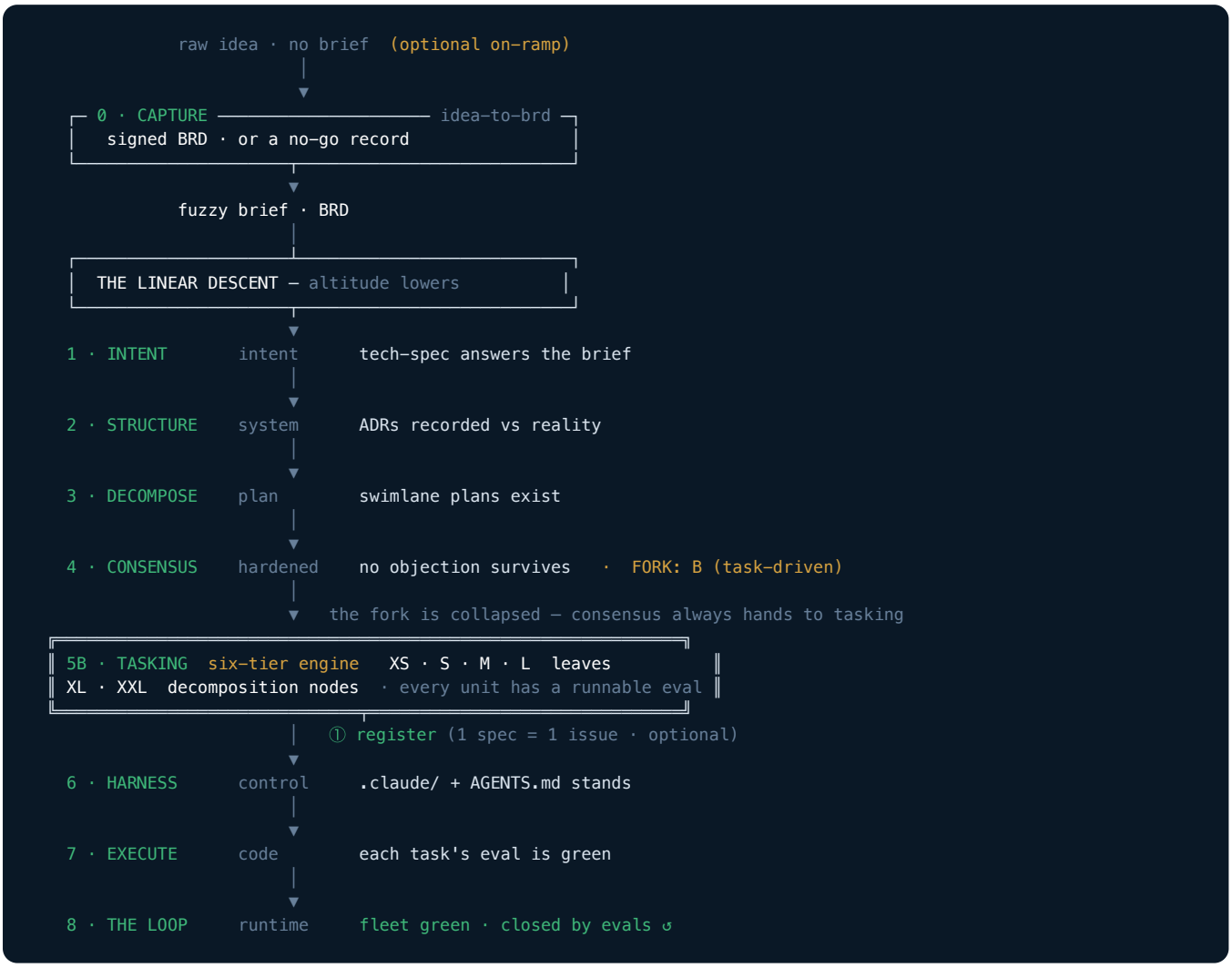

FIG 1 The 8-pass descent, now a straight line — Consensus always hands off to Tasking (the fork is collapsed), where the six-tier engine sizes each unit (XS/S/M/L leaves; XL/XXL nodes decompose further). Register is optional; Pass 8 closes the loop. Pass 0 is the optional on-ramp: raw idea in, signed brief out. One spine, top to bottom — no branch, no rejoin.

0 No brief yet? Compile the idea into one.

PASS 0 OF 8 · OPTIONAL ON-RAMP · ALTITUDE: IDEA · *receive & interrogate the raw idea*

ENGINE · CLAUDE COWORK | SKILL · idea-to-brd v0.6.0 | OUT · SIGNED BRD OR NO-GO RECORD

Not every engagement starts with a client brief. Pass 0 takes a raw idea with no document behind it and compiles it into a BRD written in the owner's voice — or kills it cheaply. It scope-checks that this is one idea, grills the owner in **frontier rounds** — each round asks every question whose prerequisites are settled, numbered, with recommended defaults, and one reply answers the round — then drafts the brief and self-reviews it against the gate. Two honest exits: a signed BRD that Pass 1 can consume, or a no-go record parked with the reasoning intact.

SCOPE-CHECK
One idea
is this one idea? Explore the environment for facts first — facts never enter a round, and never block one.

GRILL
Frontier rounds
seven branches in leverage order — pain → do-nothing test → goal → scope boundary → success → constraints → pre-mortem.

DRAFT
BRD + self-review
`docs/brd-<slug>.md` from the template, in the owner's voice — then audited against the gate before sign-off.

GATE

The pain carries a **provenance-tagged number** — `(measured)` / `(estimated)` / `(guessed)` — with at least one KPI in the owner's terms; scope in/out are non-empty; every open question has a named owner; the do-nothing test and the pre-mortem ran; the Sign-off block is signed. The gate speaks in tokens — `CHECK_BRD=PASS|FAIL|DRAFT_OK|NOGO_OK|…` — and **canonical** must sit on the Sign-off verdict line: pending or draft there never authorizes the descent. Pass 1 must not consume an unsigned brief — and a no-go here is the **cheapest kill in the whole descent**.

The frontier-rounds grill adapts Matt Pocock's `batch-grill-me`; Pass 0 adds the fixed branch map, the no-go exit, provenance tags, and the gate.

1 The client hands you the problem. You produce the spec.

PASS 1 OF 8 · ALTITUDE: INTENT · *receive & comprehend the brief*

ENGINE · CLAUDE COWORK | SKILL · brd-docs-to-tech-req v0.6.0 | OUT · TECH-SPEC

Read the BRD like a senior engineer, not a stenographer. Find the real problem underneath the ask, name who actually hurts when it is unsolved, and pin the numbers that define success. Ambiguity left here compounds through every pass downstream — this is the cheapest place in the whole spine to kill it, and the most expensive to leave alone.

The interrogation runs in frontier rounds — facts never enter a round; unanswered questions ride twice, then become owned open questions. What survives becomes a typed **gap register** (`GAP-001` · scope | definition | number | data · blocker | minor): a blocker left open fails the gate. Prior art is checked at the problem level only — never the codebase — and owners who are absent get an exported questionnaire, not silence.

UNDERSTAND
Read
the BRD like a senior engineer — the one-paragraph "solved" statement comes first: the real problem underneath, who hurts, the numbers that define success.

INTERROGATE
Grill
frontier rounds with recommended defaults; the 2–3 questions whose answers most change how this gets built; ambiguity here compounds downstream.

CRYSTALLIZE
Spec
replay the "Locked:" recap, then resolve into one technical answer — falsifiable requirements (must / should / could / won't), metrics traced to the BRD's KPIs.

GATE

You can restate the client's problem in one breath and show the tech-spec answers it — scope, data, and success metric named, every requirement falsifiable, every blocker gap closed. `--draft` validates structure but never authorizes; the canonical run passes only a spec whose Sign-off verdict line reads **canonical** — `CHECK_TECH_SPEC=PASS` is the only verdict that hands to Pass 2. Brief-in, spec-out — never blur them. And no premature technology: naming one here is the most common Pass 1 failure.

2 Greenfield or brownfield — and write the ADRs.

PASS 2 OF 8 · ALTITUDE: SYSTEM · *ground the spec, record the decisions*

ENGINE · CLAUDE CODE · ON THE REPO | SKILL · tech-req-to-adrs v0.3.0 | OUT · docs/adrs/*.md + CONTEXT.md

Before any plan, name the ground. Brownfield is a system you did not write: open it end to end, and check the spec against the real schemas, sources, and jobs you find. Greenfield is a system that does not yet exist: confirm there is nothing to inherit, then ground the spec in the source systems and constraints it must honour. Either way you leave Pass 2 knowing the terrain — what is actually there, and what the work must respect.

Not every fact earns an ADR — the **worthiness test** keeps the record honest: a decision qualifies only if it is hard to reverse, stood on downstream, and could have been otherwise. Shared vocabulary goes to `docs/CONTEXT.md`, the glossary Pass 6 wires into the harness — terms only, pinned as they crystallize.

NAME THE GROUND
Field
brownfield (a system you didn't write) vs greenfield (nothing yet) — name it explicitly; the rest follows.

GROUND
Check
open the system end to end; reconcile the spec against real schemas, sources, and jobs.

RECORD
ADRs
write `docs/adrs/*.md` — grounding decisions that pass the worthiness test (facts + constraints), never how to build. That's Pass 3.

GATE

The system is understood and its grounding decisions are recorded durably, so Pass 3 can read them without re-deriving the terrain. An ADR records what is true — schemas, joins, constraints — never how to build. The moment an ADR says "build X", you've drifted into planning. An ADR says what is true; `CONTEXT.md` says what things mean.

3 Split the work along its natural seams.

PASS 3 OF 8 · ALTITUDE: PLAN · *decomposition · break it into swimlanes*

ENGINE · CLAUDE CODE · SAME SESSION | SKILL · reqs-to-swimlane-plans v0.3.0 | IN · READS THE ADRs

Take the system understood in Pass 2 — same session, ADRs in hand — and split the work along its natural seams. Read the ADRs, then cut where the system already wants to be cut: by feature or by component. Each seam becomes one swimlane; each swimlane gets its own focus-oriented sketch plan. The common case is two lanes — transform versus serve, a medallion lane and a serving lane — but a system carrying several features splits into several lanes. The right number of lanes is the number of genuine seams. No more, no fewer.

Prefer the **highest seam, and the fewest** — cut at the published contract, above implementation detail. And before a lane earns a plan it passes the **deep-lane test**: you can say what it does, how you use it, and what it depends on — without reading its internals. A lane that can't be understood from its interface alone is a wrong boundary: move it or fold it.

DECOMPOSE
Seams
cut where the system is already jointed — a feature edge or a component edge, not an arbitrary slice.

SWIMLANE
One plan each
every seam becomes one swimlane carrying exactly one sketch plan — one lane, one plan, one focus.

GROUNDED
Reads ADRs
each lane inherits the architecture decisions in `docs/adrs/` — it builds on Pass 2, not around it.

GATE

One sketch plan per swimlane, each listing features, dependencies, and a sane build order, inheriting the ADR decisions. Plan-altitude only — no atomic tasks, no code. The session carries from Pass 2; the understanding becomes the plans without leaving the session.

REFERENCE SHAPE — THE COMMON TRANSFORM → SERVE CASE

One concrete instance of the seams Pass 3 splits into: a transformation lane (medallion: raw → bronze → silver → gold) feeding a serving lane (MCP / API over gold). Other systems split by feature or component — this is the shape, not the rule.

4 Bring a different engine to refute the plans.

PASS 4 OF 8 · ALTITUDE: PLAN (HARDENED) · *consensus · a different model attacks*

ENGINE · CROSS-FAMILY ADVERSARY (codex / kimi) | SKILL · sketch-plans-adversarial-review v0.5.0 | CLI · cvg review --a

A single model won't refute itself hard enough. So Pass 4 brings a different engine to attack the plans, one at a time, as a skeptic who defaults to "refuted." It grounds each plan against the tech-spec and the ADRs — hunting contradictions, gaps, and silent assumptions — and every objection is either FIXED in a plan or ACCEPTED with a named owner. Nothing is silently dropped. Cross-model disagreement is not noise here; it is the entire point.

When argument can't settle an objection, settle it with a **throwaway prototype** — the smallest disposable artifact that answers the question, run against real data. The result decides FIX or ACCEPT; the prototype is **evidence, not deliverable** — throw it away.

ATTACK
Refute
a different model, one plan at a time, as a skeptic who didn't write it — default to refuted.

GROUND
Drift?
check each plan against the tech-spec and the ADRs — contradictions, gaps, silent assumptions.

SHARPEN
Fix / Own
every objection FIXED in a plan or ACCEPTED with a named owner — nothing silently dropped.

GATE — FALSIFIABLE, NOT HONOUR-SYSTEM

The gate reads a **stamped objection-log artifact**, never plan prose. It passes only when: a **different-family** adversary attacked (proven by a provenance stamp + input hashes computed by the referee — not a grep of a word); every objection is FIX-or-ACCEPT-with-owner; the plans have **not drifted** since the review (re-hashed); and `FORK: B` is declared. The referee never edits — `cvg review --adversary codex,kimi` dispatches, the engine writes, `cvg review --check` gates → `CHECK_CONSENSUS=OK`. The fork is collapsed: consensus **always** hands off to tasking.

The fork, collapsed — one path: tasking

V4 · WHY PLAN-DRIVEN (A) RETIRED AND TASK-DRIVEN (B) BECAME THE WHOLE PATH

Converge used to fork at Consensus — wrap the whole system in one plan-driven spec (Fork A / SDD), or draw the boundary around each unit (Fork B / tasking). v4 **collapses** it. Frontier engines execute well-scoped atoms reliably, and a tree of verified atoms composes back up more safely than one monolith — so there is no need for a separate spec-driven paradigm. Consensus **always** hands off to tasking; a six-tier engine sizes every unit from a one-liner to a whole backbone.

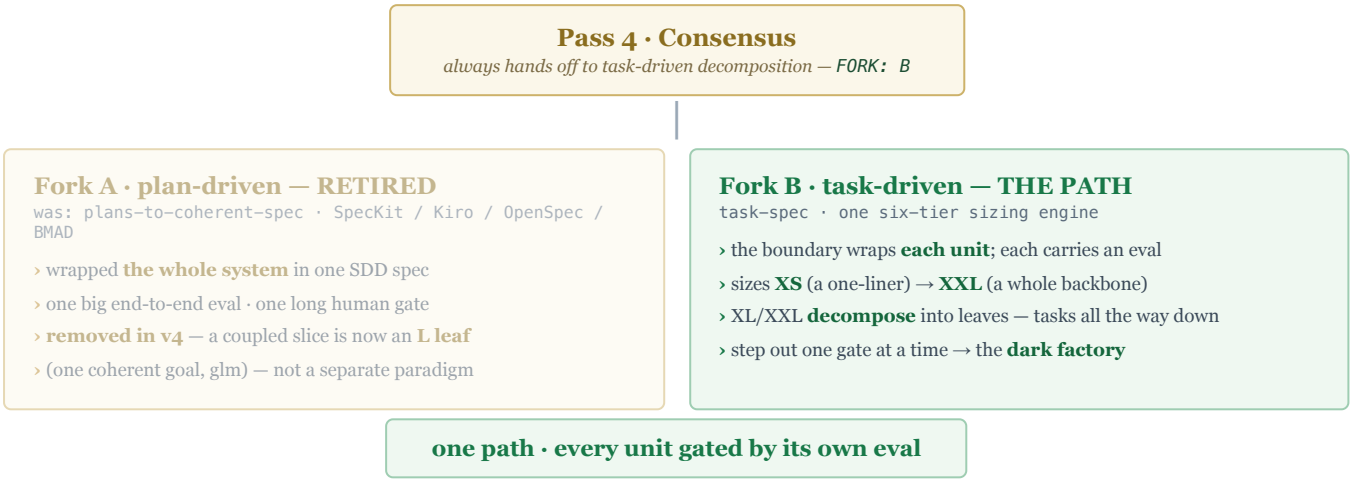

FIG 2 The fork is a constant now: Consensus always stamps `FORK: B` and hands to Tasking. Fork A is retired — the six-tier engine absorbs the coupled-work case as an L leaf, so there is no second paradigm to route out to.

THE SIX-TIER ENGINE ABSORBS THE RANGE

Leaves run · nodes decompose

Leaves (XS/S/M/L) are runnable atoms — each fits one fresh context window and verifies as one PR / test-suite; a gate-checked **write-surface budget** (XS≤1 … L≤5 paths) catches a mis-sized unit. **Nodes** (XL/XXL) are not runnable — they MUST declare `children:` and expand into leaves. A worker dispatches the children, never the node — `cvg tasks gate` refuses a node.

WHY ONE PATH IS SAFER

Recursion, not escalation

Big work used to route **out** to SDD. Now it decomposes **in**: the same task-spec format, self-similar at every scale, so the seam contracts are written once and per-unit tasks can't drift. Research (clispec, Moai, Thread AI, AgentSpec — 2026) put the format at or above SOTA on every axis.

TASKS ALL THE WAY DOWN

The dark factory needs one unit model at every scale. A subjective slice becomes a human-checkpoint eval; a coupled slice becomes an L leaf; a whole backbone becomes an XXL node that decomposes to a tree of leaves. There is no route out — the parent composes its children's verified results back up.

5B The six-tier engine — size every unit, from XS to XXL.

PASS 5B · THE SIZING MODEL · TASK-SPEC V3.4 · *one engine, a one-liner to a whole backbone*

ENGINE · TASK-SPEC (SIX-TIER) | SKILL · task-spec v3.4.0 | GATE · cvg tasks validate

Every unit gets a t-shirt size — sized the agent-native way: not by human hours, by **context, verification, and blast radius** (the 2026 SOTA signal — Young, Anthropic, Vaughan, Vest). Two kinds. **Leaves** (XS/S/M/L) are runnable atoms that fit one fresh context window and verify as one PR / test-suite. **Nodes** (XL/XXL) are decomposition directives — not runnable; they expand into leaves. The write-surface budget is gate-checked, so a mis-sized leaf is caught before it ships and a node with no children is refused.

SIZE	KIND	WRITE BUDGET · BACKEND	EXAMPLE
XS	leaf	≤ 1 path · Kimi	a config key; bump a dependency
S	leaf	≤ 2 paths · Kimi	a `/health` endpoint + its test
M	leaf	≤ 3 paths · Kimi	a dbt model + schema + test
L	leaf · ceiling	≤ 5 paths · **glm** required	a service slice as ONE coherent goal
XL	node · decomposes	children ≥ 2 (owns none)	a whole swimlane → its leg leaves
XXL	node · decomposes	children ≥ 3 (owns none)	a whole backbone → swimlane nodes

THE RECURSION — THIS IS THE WHOLE IDEA

XXL backbone → XL swimlanes → M/S/L leg atoms → XS units. A worker runs the **leaves** and composes their verified results back up the tree. A budget that gets breached means the decomposition was too coarse — the signal to split further. Big work decomposes **in** (recursion), it never routes **out** (escalation).

GATE — CVG TASKS VALIDATE

Sizing is enforced, not advisory: a leaf over its write-surface budget WARNS "mis-sized — split or reclassify UP"; an XL/XXL node with too few children FAILS ("must decompose — no route out"); L requires `execution_backend: glm` + one coherent done-condition. The format is vendor-neutral and self-verifying — the same file runs on Claude, Codex, or Kimi.

5B Tasking, in practice — a backbone into eval-backed units.

PASS 5B OF 8 · ALTITUDE: ATOMIC UNIT · *the plans decompose into task-specs*

ENGINE · TASK-SPEC SKILL | SKILL · task-spec v3.4.0 | CLI · cvg tasks validate | gate | dod

Consensus hands off here, always. The hardened swimlane plans decompose into **eval-backed task-specs** — the trust boundary wraps each unit, and verification is per-unit. Trust is earned one unit at a time: each eval that goes green is a gate you no longer hold by hand. This is the path to the Dark Factory — many small, automatable gates a fleet can close on a schedule. On the proving ground, the whole analytics backbone is already tasked: **9 legs → 9 task-specs**, wired into a dependency DAG, every one `validate` ok · `gate` DELEGATE · `dod` COMPLETE.

v3.4 made the cut recursive: each unit is a **vertical slice** (a tracer bullet, not a horizontal layer) sized by the six-tier engine; behaviors (`B-1`, `B-2`…) trace to evals and back; and the seam contracts hardened at Pass 4 ride in as guardrails, so a per-unit task can't drift from the consensus.

SIZE
Six tiers
XS/S/M/L leaves run; XL/XXL nodes decompose. The write-surface budget catches a mis-sized unit at the gate.

BIND EVAL
Done = ?
a runnable eval defines done — the test passes, the query returns N, the probe reaches a served answer.

TRACE
DoD matrix
`cvg tasks dod` renders every behavior → its verifying eval → result; an untraced behavior FAILS the DoD.

GATE — `CVG TASKS GATE` (SAFE-TO-DELEGATE)

Every task carries a runnable eval — no eval, not a task yet. The delegation gate runs two stages — structural + sizing validation, then a broken-eval guard — and **refuses to delegate an XL/XXL node** (a worker dispatches its children). Sign-off is a key-optional HMAC seal, **tamper-evident** — it proves the evals were not weakened after sign-off. Self-contained and vendor-portable: Claude, Codex, Kimi, or manual.

① Each Task-Spec becomes one tracked issue.

REGISTER · *the tracker as state*

SKILL · task-specs-to-issues v0.3.0	--tracker linear (github | jira)	OUT · 1 SPEC = 1 ISSUE

When tasks leave Tasking, you optionally register each Task-Spec as exactly one tracker issue — its state shadow. The 1:1 mapping is the whole mechanic: one spec, one issue, no drift. The tracker is a **pluggable backend** behind a five-verb adapter — `preflight · upsert · link · list-ready · write-result` — so GitHub Issues, Linear, or Jira are interchangeable slots, not commitments. `blocked-by` links carry the dependency graph: `42 blocked-by 41` puts the build order into the tracker itself, so state can be read instead of inferred. `register.sh --dry-run` detects `depends_on` cycles before any write; the registration gate requires count(issues) == count(signed-off specs), no orphans, no cycles. And the eval travels onto the board — the spec's Exit Check is copied verbatim into the issue body as the close condition.

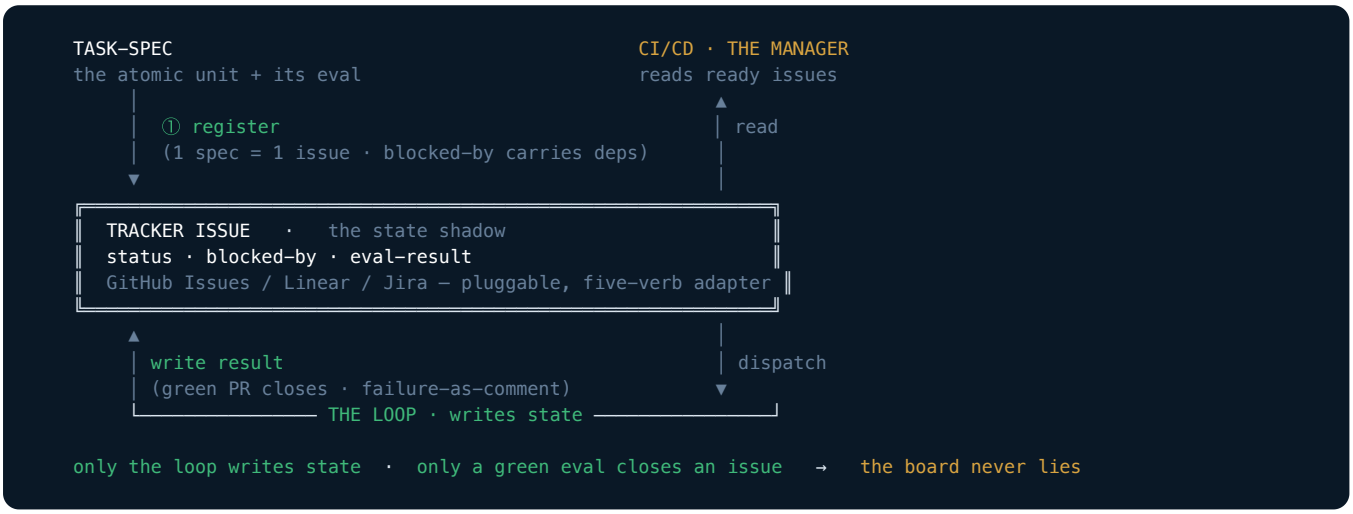

FIG 3 The tracker is the shared state both loops read and write — one Task-Spec registers as one issue; CI/CD reads ready issues, the loop writes results. Only a green eval closes an issue, so the board is state you can dispatch against unattended.

This is what makes the board state **perfect**. Only the loops write state — never a human nudging a card. Only a green eval moves an issue to done — never a feeling, never an attempt-count. Because every transition is mechanical and eval-gated, the board never lies: what it shows is what is true. A "done" column you can trust is the precondition for the Manager loop dispatching against it unattended.

> REPO-VS-TRACKER IS A REAL CHOICE, NOT AN UPGRADE
>
> Stay at task-spec (repo files) when you want repo-local control — the specs travel with the code, version with the commit, need no external system. Add task-specs-to-issues when you want a board the loops read across sessions and machines. The tracker is that shared memory; the repo files are the source.

6 Build the rails the work rides on — the quality moat.

PASS 6 OF 8 · ALTITUDE: CONTROL LAYER · *harness · generate the build-system the stack needs*

ENGINE · AGENTS-KBS-TECH-STACK | SKILL · stack-to-harness v0.3.0 | OUT · .claude/ + AGENTS.md

Pass 6 binds the harness to the system's tech stack — not to the tasks in flight. The dbt knowledge base, the MCP server agent, the DuckDB ruleset exist because the system runs dbt, MCP, and DuckDB — not because ISSUE-41 or ISSUE-42 named them. The tasks scope which stack components are in play, so you scaffold only what the system actually uses, never speculatively. The stack fills the harness; the tasks point at it. Tasking precedes Harness for exactly this reason: the work reveals which slice of the stack is in scope.

The control plane is verified, not hoped for: `quality-gate.sh --strict` exits 7 on a blocker. Cross-tool mirrors (`AGENTS.md`, Cursor, Copilot) never clobber hand-edits — a changed mirror ships as a `.proposed` sibling. The `CONTEXT.md` glossary is wired in, and **a manual step deserves a wizard, not prose** — interactive scripts under `.claude/scripts/`.

SCOPE
Read stack
the tasks reveal which techs are in play (dbt + duckdb + mcp) — scaffold exactly those, not speculatively.

SCAFFOLD
Agents + KBs
paired architect + developer per tech, grounded in real docs, plus KBs and rules.

EMIT
Cross-tool
`AGENTS.md` · Cursor · Copilot — every engine inherits one contract.

GATE

The control plane stands and is grounded in the system's tech stack — the tasks scope which components are in play, the stack fills the content. This is the quality moat: the files that make any commodity agent succeed, and the one artifact that outlives the project.

8 Build the execution loop. Schedule the Manager later.

PASS 8 · THE LOOP — *loop engineering*

Loop Engineering is the discipline of designing the loops that prompt agents, rather than prompting agents. The unit of work is no longer the message but the cycle. One primitive recurs everywhere — take an action, read the feedback the environment returns, decide the next move, repeat until the **goal condition** holds. Convergence is defined by the goal, never by attempt-count: a loop that stops because it tried three times has not converged; a loop that stops because the eval is green has.

Pass 8 · The Loop — task-loop (the execution loop). This is the loop you build. It takes ONE issue (`--issue N`, handed to it by a human or by CI — you never pick the task), then **READ**s its task-spec plus the cited ADRs and the grounded harness, **ACT**s on a branch inside `touches_paths`, and runs the task's own **EVAL**. RED: the exact failure feeds back into a tight local loop, bounded by `budget_iterations`. RED twice with the same failure: stop patching, start diagnosing — reproduce minimally, hypothesize, verify, fix once. GREEN: **SETTLE** opens a Pull Request that closes the issue.

FIG 4 The execution loop — task-loop takes one issue, writes code, runs the task's eval, feeds RED back until GREEN, then opens a PR. The Manager that picks which issue and fans out is a future CI/CD concern (GitHub Actions), not an in-session skill.

GATE

Pass 8 — the task's eval is green; every issue closed by its OWN eval, none by hand, none by attempt- count. Schedule this loop in CI/CD and the human steps out: that is the Dark Factory, one issue at a time.

The operating model — five beats per pass.

HOW THE METHOD BUILDS ITSELF · TRACK R CURRENT · TRACK M PARKED

Converge is built with its own method. **Track R** validates the method one pass at a time, and each pass is a complete vertical slice of five beats — **in order, never bundled**. **Track M** — the execution machine, fixture-tested — waits at Milestone 0 and resumes at/around R8.

BEAT	WHAT HAPPENS · WHERE IT LANDS
1 · UNDERSTAND	read the pass's skill end to end, compare against the field, deliver a hardening list — analysis only, no edits
2 · RUN	execute the pass **for real** on the proving ground; the owner judges every output
3 · CANONIZE	iterate skill + artifact until the owner calls it canonical — blueprints are **canonical v0**; a later pass may send a retro-edit back
4 · IMPLEMENT	the `cvg` subcommand for this pass — frame → dispatch → collect → gate, thin by design
5 · PROVE	re-run the pass through the CLI on the same input; artifacts and gate verdicts must match the golden manual run — **nothing to trust, only to diff**

THE TEACH BEAT — INSIDE THE C-BEAT

Closing CANONIZE includes the TEACH beat: run `pass-to-lesson` on the just-canonized artifacts — every component taught: what it is, why it is shaped this way, the decision it encodes, what breaks downstream without it — `check-lesson.sh` green, the lesson committed under `docs/lessons/`. **A pass is not closed until its lesson exists.** Rule 0 is the stance: go slow and teach — the owner's understanding is a deliverable equal to working code. The companion explains decisions, never reopens them.

ROUNDS · WHERE THE METHOD STANDS

Rounds are named `R<pass>.<beat>` — R0.U … R8.P. Status, 2026-07-21: **Passes 0 through 5B closed** on the proving ground. Capture / Intent / Structure / Decompose signed canonical (byte-parity CLI proofs); Consensus reached on a **real cross-family adversary** (kimi) that sharpened the seams — and the plan-vs-task **fork collapsed** to a single tasking path; the whole backbone is tasked (9 specs, all gated + DoD-complete). The `cvg` CLI is at **v0.10.0**, agent-native SOTA. Next: **Pass 6 · Harness** then **Pass 8 · The Loop** — where a task runs to green.

The knowledge home — the cvg/ workspace.

FIVE FOLDERS, ONE LIFECYCLE EACH

Every consuming project gets one Converge home — `cvg/`, named for the CLI — with `INDEX.md` as the front door. Artifacts flow one way and never backward: **brain feeds → docs agree → sketch explores → tasks execute → receipts prove.**

FOLDER	KIND	LIFECYCLE
`brain/`	user-generated inputs	raw, append-only, never gated
`docs/`	consensus artifacts	gated, permanent, never deleted — BRD · tech-spec · `adrs/` + `CONTEXT.md` · `lessons/`
`sketch/`	drafts	transient — sharpened at Pass 4, decomposed into tasks at 5B
`tasks/`	sealed execution units	HMAC-stamped, tracked
`receipts/`	evidence — gate verdicts, pass receipts	write-once

ONE HOME

Per project

the workspace is named for the method's CLI, not the tool of the moment; dot-space (`.cvg/`) stays reserved for machine config.

ONE DIRECTION

Never backward

inputs feed, agreements gate, drafts expire, units seal, evidence freezes — nothing flows upstream.

ONE TRUTH

The repo

second-brain tools pull from the repo, never the reverse — the repo stays the single source of truth.

THE CONVENTION

The repo stays the single source of truth; second-brain tools pull from it, never the reverse. `.cvg/` is reserved for machine config. `MAP.md` answers "where do I look for X" in ten lines, and `INDEX.md` is the workspace's front door.

The cvg CLI — a referee at 2026 SOTA.

REFEREE, NEVER A PLAYER · AGENT-NATIVE · V0.10.0

Every pass is gated by one CLI — `cvg` — wrapping the proven skill scripts. It holds **zero model credentials** and makes no LLM calls: it frames prompts, dispatches engine CLIs headlessly (codex / kimi), and gates their output. Wrapped gates are **byte-exact pass-throughs**; bash-3.2-safe, stdlib-only, dep-free in CI. Every gate ends in ONE stable, greppable token and exit codes are contracts — a harness never parses prose. Passes 0→5B are all live and closed on the proving ground.

```
# the whole 8-pass chain — one name, one token each
cvg capture   → CHECK_BRD=PASS          cvg decompose → CHECK_PLAN=OK
cvg intent    → CHECK_TECH_SPEC=PASS    cvg review --adversary codex,kimi → REVIEW=OK
cvg structure → CHECK_ADR=OK            cvg review --check → CHECK_CONSENSUS=OK
cvg tasks validate | gate | accept | dod        cvg doctor → DOCTOR=OK

# agent-native SOTA — wrap ANY command in one uniform envelope
$ cvg --json capture
{ "ok":true, "command":"capture", "token":"CHECK_BRD=PASS",
  "exit_code":0, "changed":false, "error":null,
  "meta":{ "schema_version":"1.0", "cvg_version":"0.10.0" } }

cvg agent-context  → command tree + exit-code taxonomy (retryable), as JSON
cvg <mutation> --dry-run  → reports intent, changes nothing (changed:false)
```

FIG 5 The referee surface. Default output is byte-parity prose ending in a token; `--json` wraps any command in a uniform `{ok, data, error, meta}` envelope without touching the default path; `agent-context` is the machine-readable manifest. Measured at or above the 2026 agent-native-CLI SOTA.

AGENT-NATIVE · 2026 SOTA

--json · agent-context · dod

Benchmarked vs clispec.dev / cli-agent-spec + 3 more: a uniform `--json` envelope on every command, an exit-code taxonomy with `retryable/side_effects`, a self-describing manifest, `--dry-run` on mutations, and the DoD/traceability view — all shipped.

PROVING GROUND · TRACK R

tests/uc-analytics

greenfield e-commerce Postgres, seed 42. **Passes 0→5B closed here**: BRD / tech-spec / ADRs / swimlanes signed, a real cross-family adversary (kimi) sharpened the seams to `CHECK_CONSENSUS=OK`, and the backbone is tasked — 9 specs, all `gate` DELEGATE · `dod` COMPLETE.

DOGFOODING — THE MACHINE IS BUILT BY THE METHOD

Every `cvg` subcommand reproduces the canonical manual run — artifacts and gate verdicts diffed against the golden reference — before it ships. Gates ship as versioned exit contracts whose regression suites prove the negatives fail: a fixture that wrongly PASSes on the previous version is caught, because evals must discriminate. The result is a referee you can point an autonomous engine at — it never bluffs a pass.

Why this is defensible

TWO MOATS · POSITIONING · THE HONEST BOUNDARY

The two moats

Converge holds two moats, not one, and the discipline is to keep them separate.

OPTIONALITY

Task-Spec

the eval — not the agent — defines done, so you swap Codex / Kimi / Claude freely; the coding agent is a commodity slot.

QUALITY

Harness

the control-plane files that make any commodity agent succeed; not downloadable, built over thousands of hours.

TOGETHER

Defensible

run on commodity models, produce defensible output. Two moats — optionality and quality — not one.

Positioning vs the field

The canonical SDD pipeline is Specify → Plan → Tasks → Implement, punctuated by human checkpoints — Spec Kit, Kiro, OpenSpec, BMAD all share this shape, and their gates and loops keep growing. Converge's claim is not a feature checklist but a **difference in contract**: the adversarial **Consensus** pass, where a **different-family** model attacks each plan default-to-refuted; the **collapsed fork** — one six-tier tasking engine that decomposes big work **in place** rather than routing out to a second paradigm; and the closed **Loop** of Pass 8, where every unit is born with a runnable eval and only a green eval closes an issue. The named SDD frameworks solve the coupled-spec case Converge now folds into a single **L leaf** — the bet is that recursion beats a fork.

THE HONEST BOUNDARY

The Loop converges to green evals, not to correct outcomes. A passing spec test guarantees only that the code matches the spec — if the spec is wrong, the code faithfully implements the wrong thing. The Dark Factory automates the build, not the judgment of whether the spec was right. The defense is structural and partial, not total: Pass 1's gate forces the spec to answer the brief, and Pass 4's adversary attacks it before any code is written. Stating this plainly is the point — the intellectual honesty is the moat's foundation, not a footnote.

The runtime, proven

WHAT THE SKILLS ALREADY ENFORCE — NOT A PROMISE, A MECHANISM

The gates above are not honour-system prompts. The task-spec skill ships the runtime that makes "done = proved" a mechanism a human cannot fake and an agent cannot shortcut. Five enforcement layers matter most, and all five already exist on disk.

LAYER	WHAT IT ENFORCES · WHERE IT LIVES
Atomic state machine	A locked, portable transition protocol — advisory lock (mkdir-based, macOS-safe), read current status, validate the transition against a real state machine, write via tmp-file + atomic move. Two agents cannot both claim one ready task. `transition-status.sh`
Tool-captured evidence	The acceptance gate re-runs the evals itself — it never trusts the executor's word. A non-zero exit is rejected automatically. This is the hole DGE's "evidence gate" fills; Converge fills it too. `accept-task.sh` · GATE A
Anti-reward-hacking	Beyond re-running: blast-radius check (changed files must be inside `touches_paths`), HMAC contract integrity (evals were not weakened after sign-off), and gold-sanity (the eval must FAIL on the unpatched baseline). GATE B · C · E
Rebuildable ledger	The T-spec frontmatter is truth; `_metrics.jsonl` is an append-only ledger; `_state.yaml` is a rebuildable index. The board is a projection of one source of truth — it cannot silently diverge. `rebuild-state.sh`
Teaching gate	The lesson is enforced, not hoped for: every artifact taught, every decision naming a rejected alternative, every term of art defined, 3–5 check-yourself questions, and a clean tree. A pass is not closed until its lesson exists. `check-lesson.sh`

Field-proven 2026-07: v0.1 released (MIT, stack-agnostic scrub, 13/13 skills validated; task-spec suites + L0/L1/L2 conformance green). Milestone 0 floor green — 6/6 sealed specs stamped DELEGATE. Passes 0 and 1 closed end-to-end on the proving ground — signed canonical, lessons taught, CLI runs byte-identical to the human gates.

THE ONE GAP WORTH CAPTURING FROM THE FIELD

Converge's eval verdict is binary — green or red. A useful third state exists that it does not yet model: **ambiguous** — the eval ran but cannot prove the claim. Adding an `ambiguous` verdict that never satisfies a gate is the single cheapest, most honest upgrade to the acceptance contract. It is atomic, falsifiable, and Task-Spec-shaped today.

WHAT THIS MEANS FOR AUTONOMY

Autonomy is the reward for understanding, not a substitute for it. Converge earns the right to run dark at Pass 8 by spending human judgment at Passes 1–4 — the swimlanes, the ADRs, the adversary. A running loop with no upstream comprehension builds the wrong thing flawlessly. The factory must run dark and be aimed correctly; these passes are what aim it.

One method, every engineer

THE ROLE LIVES IN THE EVAL, NOT THE LOOP

The loop never inspects what a task is — it inspects only one thing: does the task carry a runnable eval, and is that eval green. All role-specificity is pushed down into the per-task eval. The loop primitive — take an action, read the environment, decide, repeat until the goal condition holds — is identical whether the artifact is a SQL model, a service, an IaC module, or a retriever. That is precisely why Converge is canonical across disciplines: the role does not change the machine, it only changes the assertion the machine waits on.

ROLE	WHAT ITS TASK'S EVAL IS
Data Engineer	a dbt test + a control-sum query against gold
Software Engineer	a unit / integration test that passes
DevOps	`terraform plan` clean + a smoke test
AI Engineer	a retrieval-precision or eval-set threshold met

> THE PRINCIPLE
>
> The role lives in the eval, not the loop. `task-loop` runs whatever until its eval is green — and CI/CD schedules it — both role-agnostic by construction. That is why one method serves Data Engineering, Software, DevOps, and AI Engineering alike.

The method, end to end on one feature

POSTGRES → DUCKDB → DBT → MCP · TASK-DRIVEN

One feature carries the whole framework: **"daily revenue per product category, served via MCP so a non-engineer can ask for it."** The real metric is `SUM(payments.amount) WHERE payments.status='paid'`, grouped by `products.category` and `date(orders.ordered_at)` in UTC — every pass on the real repo. This feature is the proving ground itself — `tests/uc-analytics`, Postgres on 5433, deterministic seed 42.

PASS	WHAT HAPPENS ON THE REAL REPO
P0 · Capture	Raw idea → `docs/brd-analytical-backbone.md` in the owner's voice, signed 2026-07-19 — the first canonical artifact born on the proving ground. *Gate: `check-brd.sh` green.*
P1 · Intent	BRD → tech-spec: one gold model + one MCP tool over raw orders + payments + products. *Gate: the spec answers the brief.*
P2 · Structure	Brownfield. Open `src/db/01_schema.sql`; write ADRs `0001-join-on-order-id`, `0002-utc-date-grain`, `0003-revenue-is-paid-only`. System understood vs reality.
P3 · Decompose	Two plans, plan-altitude only: Plan A transform (silver → `gold_daily_revenue`), Plan B serve (mcp tool `query_daily_revenue`).
P4 · Consensus	`cvg review --adversary codex` attacks: *"midnight off-by-one (no TZ)? refunds counted?"* Fix: pin UTC (ADR-0002). Accept: refunds out of v1. **Hands off to tasking** (the fork is collapsed → `FORK: B`).
P5B · Tasking	`tasks/build_gold_revenue` (eval: dbt test + control-sum) and `tasks/build_mcp_revenue` (eval: tool result == gold query). Each born with its eval.
① Register	`--tracker linear`: build_gold_revenue → ISSUE-41 [ready]; build_mcp_revenue → ISSUE-42 [blocked-by 41]. The board is shared state.
P6 · Harness	Stack = dbt + duckdb + mcp → scaffold `.claude/` with dbt-architect, mcp-developer, KBs, `AGENTS.md`. The control plane stands. This is the quality moat.
P8 · The Loop	`task-loop --issue 41`: dbt test RED (null categories) → add `COALESCE` → GREEN + control-sum matches → PR; 41 done. Unblocks 42 → eval green → PR; 42 done.

THE DARK FACTORY, ON THIS REPO

A non-engineer asks the MCP *"revenue by category yesterday?"* — and it answers. Built brief → deploy, every step eval-verified, the human walked out at the gate. That last step is the Dark Factory, running on this repo.